

ECOLE ITAG TEMARA – ETABLISSEMENT PRIVE

Autorisation n° : 4/09/01/94

245,AV Moulay Ali Chérif Massira 1 - Temara

E-mail: itagtemara2020@gmail.com

Site web: www.groupeitag.net

Projet de Fin Formation

< < Gestion de Stock >>

Filière :

Technicien Spécialisé en

<<Développement Informatique >>

Encadré par : M. Mustapha Laghzil

Réalisé par :

NOM ET PRENOM : AMINE JEBBOUJ

REMERCIEMENTS

Je tiens à remercier monsieur le directeur Mr. NACER et toute l'administration du centre ITAG qui nous a offrir un entourage organisé durant toute au long de l'année scolaire, ainsi que mes professeurs et encadrants pour leurs efforts, leurs conseils et leur accompagnement tout au long de cette formation.

J'adresse aussi mes sincères remerciements au Mr. LAGHZIL MUSTAPHA, mon professeur pour ses efforts ; ses conseils et ses orientations lors de l'apprentissage.

Dédicace

Je dédie ce travail à ma famille et mes proches qui m'ont toujours soutenu et encouragé et qui m'a toujours soutenu et de le pousser vers le mieux. C'est grâce que d'après elle je puise de l'énergie pour continuer.

Merci à moi pour tout l'effort effectuer et de cet avancement et l'amélioration depuis mon enfance.

TABLE DES MATIERES :

1. Remerciements	1
2. Dédicace	2
3. Introduction générale	7
4. Présentation du projet	8
5. Problématiques	9
6. Solutions proposées	10
7. Gestion de projet	11
a. Diagramme de PERT	12
b. Diagramme de Gantt	13
8. Partie Analyse et Conception	14
a. Méthode Merise	15
b. Dictionnaire de données	16
c. Règles de gestion	17
d. Model conceptuel de données(MCD).....	22
e. Model Logique de données(MLD).....	23
f. Model Physique de données (MPD)	24
g. Méthode UML	25
h. Cas d'utilisation	26
i. Diagramme de classes	27
j. Diagramme de séquence	28
9. Technologies utilisées	29
10. Démonstration	30
11. Résultats et avantages	41
12. Conclusion et perspectives	42
13. Webographie	87

■ Liste des figures :

Figure 1 : Tableau diagramme de PERT	11
Figure 2 : diagramme de PERT	12
Figure 3: Diagramme de GANTT	13
Figure 4 : Dictionnaire des donnés	15
Figure 5 : modèle conceptuel de donnes.....	21
Figure 6: modèle logique de donnes.....	22
Figure 7: modèle physique de donnes.....	23
Figure 8 : Le diagramme de cas d'utilisation.....	25
Figure 9 : Le diagramme de classe	26
Figure 10 : le diagramme de séquence	27
Figure 11 : Interface – Page Login	30
Figure 12 : Interface – Menu principal.....	32
Figure 13 : Interface – Utilisateur	33
Figure 14 : Interface – Produit	48
Figure 15 : Interface – Lot	49
Figure 16 : Interface – Vente	74
Figure 17 : Interface – Inventaire	62
Figure 18 : Interface – Ligne de vente	63
Figure 19 : Interface – Ligne de commande d’achat.....	64
Figure 20 : Interface – Ligne de stock.....	66
Figure 21 : Interface – Ligne d’inventaire.....	67
Figure 22 : Interface – Commande d’achat.....	76
Figure 23 : Interface – Catégorie.....	77
Figure 24 : Interface – Fournir.....	79
Figure 25 : Interface – Fournisseur.....	78
Figure 26 : Interface – Bon de livraison	75
Figure 27 : Interface – Entrepôt.....	80
Figure 28 : Interface – Emplacement.....	81
Figure 29 : Interface – Audit	82
Figure 30 : Interface – Rôle	82
Figure 30 : Interface – Rôle	82
Figure 31 : Interface – Paiement	65

Liste des Tableau :

Tableau 1 : Tableau de PERT.....	11
Tableau 2 : Dictionnaire des données	15
Tableau 3 : Table Client	15
Tableau 4 : Table Produit	15
Tableau 5 : Table Catégorie	16
Tableau 6 : Table Entrepôt	16
Tableau 7 : Table Ligne_Achat	16
Tableau 8 : Table Commande_Achat	16
Tableau 9 : Table Ligne_Vente	16
Tableau 10 : Table Vente	17
Tableau 11 : Table Fournisseur	17
Tableau 12 : Table Ligne_Stock	17
Tableau 13 : Table Emplacement	17
Tableau 14 : Table Lot	17
Tableau 15 : Table Inventaire	17
Tableau 16 : Table Bon_Livraison	18
Tableau 17 : Table Utilisateur	18
Tableau 18 : Table Rôle	18
Tableau 19 : Table Audit	18
Tableau 20 : Table Paiement	19
Tableau 21 : Table Ligne_Inventaire	19
Tableau 22 : Table Fournir	19

Introduction générale :

La gestion des stocks est une fonction vitale pour toute entreprise.

Une mauvaise organisation entraîne des pertes financières, des erreurs dans les ventes et une difficulté de suivi des produits.

Ce projet s'inscrit dans le cadre du programme de fin de formation en développement informatique à l'ITAG.

L'objectif est de concevoir une application de bureau permettant une gestion efficace des produits, clients, fournisseurs, ventes, achats et inventaire, tout en assurant la traçabilité grâce à des procédures stockées et des déclencheurs d'audit SQL.

Présentation du Projet :

Notre projet consiste en la conception et la réalisation d'un **système d'information de gestion de stock**. Ce système permet une gestion intégrée des différents acteurs et ressources liés au stock : produits, clients, fournisseurs, ventes, achats et inventaires. L'application développée est une **solution Desktop en C# WinForms**, connectée à une base de données **SQL Server**, qui offre :

- La gestion complète des opérations CRUD (ajout, modification, suppression, consultation).
 - La centralisation et la sécurisation des données.
- L'intégration de procédures stockées et de triggers pour assurer la **traçabilité et la fiabilité** des opérations.
 - Une interface simple et intuitive pour faciliter l'utilisation par les différents profils (administrateur, magasinier, responsable).
- La conception repose sur la méthode **Merise** pour la modélisation des données et sur **UML** pour la modélisation fonctionnelle. Ce projet met en valeur l'importance des technologies Desktop dans le domaine de la gestion de stock et propose une solution pratique, évolutive et adaptée aux besoins des entreprises.

Problématiques :

La gestion manuelle des stocks entraîne plusieurs difficultés :

- Risque d’erreurs dans les ventes et achats.
- Mauvaise coordination entre les différents acteurs.
 - Perte ou duplication des données.
 - Absence de traçabilité des opérations.

Solutions Proposées :

Pour répondre à ces problématiques, le système développé :

- Centralise toutes les données dans une base SQL Server.
- Permet un accès rapide et sécurisé via une interface C# WinForms.
 - Optimise la gestion des ventes, achats et inventaire.
- Intègre un système d'audit automatique pour garantir la traçabilité

Gestion de projet :

Planification de projet :

La gestion de projet de Stockage est un ensemble de méthodes et de pratiques permettant de planifier, organiser, coordonner et contrôler efficacement les ressources, les délais et les activités, dans le but de réaliser un objectif précis tel que l'amélioration des soins, l'optimisation des processus internes, ou l'implémentation d'un nouveau système de Stockage ou administratif.

1) Diagramme de PERT.

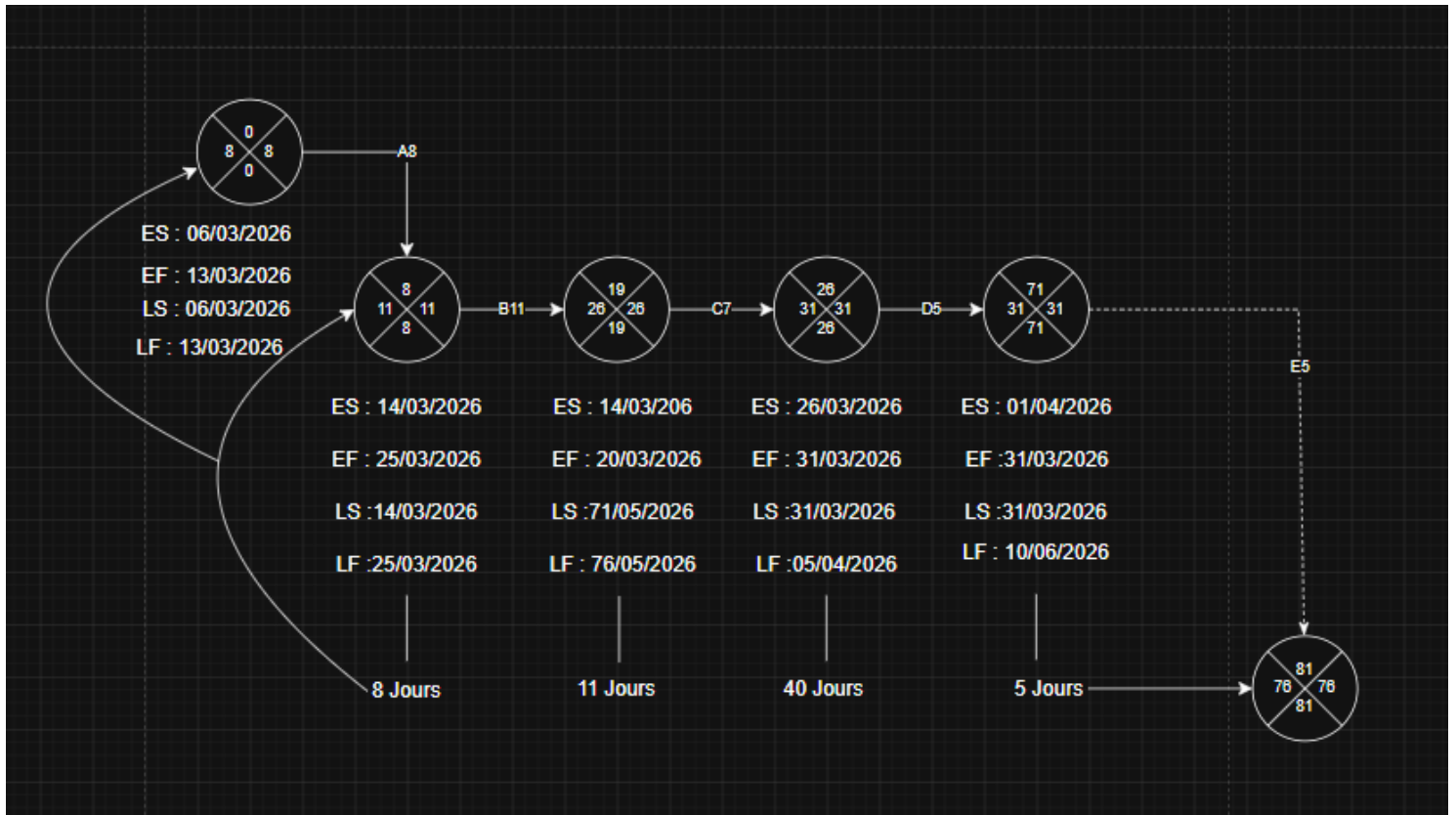
Le diagramme de PERT est un outil de gestion de projet qui permet de représenter les différentes tâches à réaliser, leurs enchaînements et les dépendances entre elles, sous forme de réseau (graphe), pour mieux planifier et contrôler le projet.

La figure à dessous illustre la planification des taches, qu'on a prévues durant la période du rapport.

Voici tableau de PERT de ce projet :

Id	Tache	Durée	Antériorités
A	Recherche de projet	8	Aucune
B	Cahier de charge	11	A
C	Conception	7	A
D	Choix de Tech	5	B
E	Réalisation	40	D
F	Test	5	C
G	Rapport	5	A

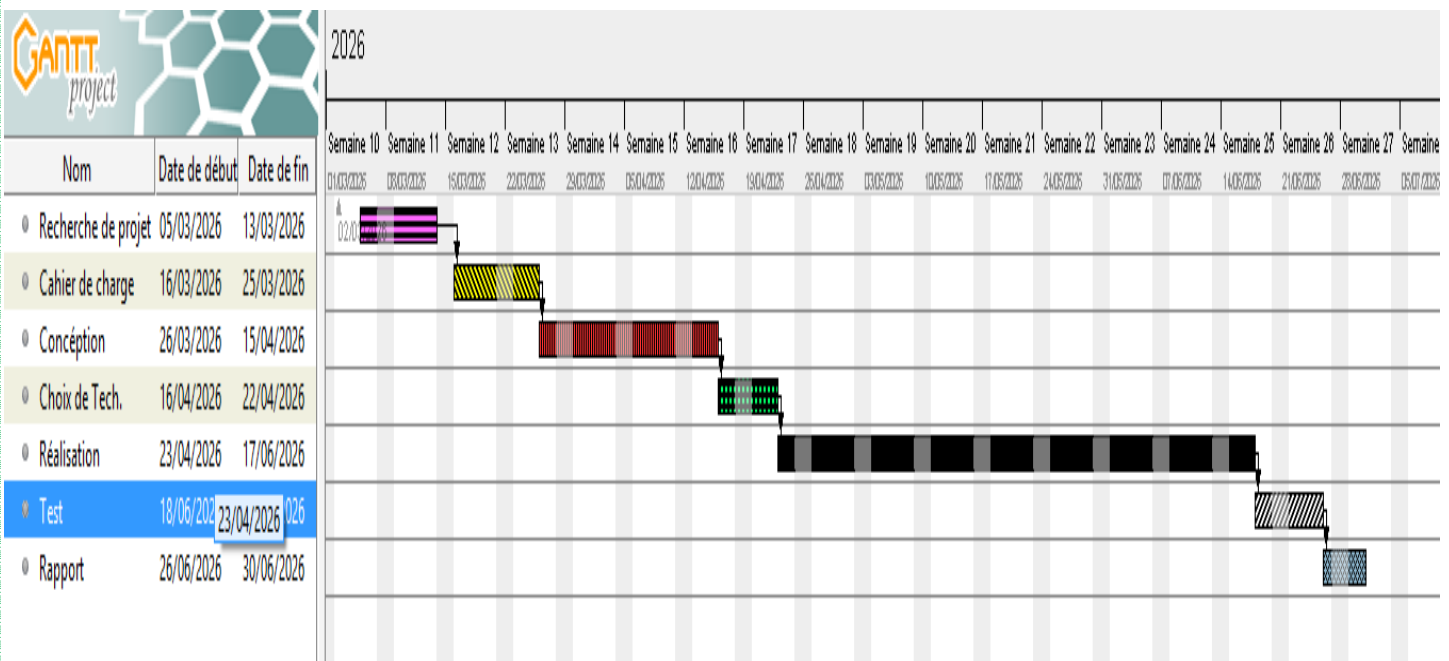
Voici diagramme de PERT de ce projet :



2. Diagramme de GANTT.

Le diagramme de Gantt est un outil graphique qui permet de représenter visuellement l'enchaînement et la durée des tâches d'un projet, sous forme de barres horizontales sur une ligne du temps.

Voici diagramme de GANTT de ce projet :



Partie Analyse et conception :

A. Besoin la méthode :

La conception d'un système d'information n'est pas évidente car il faut réfléchir à l'ensemble de l'organisation que l'on doit mettre en place. La phase de conception nécessite des méthodes permettant de mettre en place un modèle sur lequel on va s'appuyer. La modélisation consiste à créer une représentation virtuelle d'une réalité de telle façon à faire ressortir les points auxquels on s'intéresse.

Ce type de méthode est appelé analyse. Il existe plusieurs méthodes d'analyse, la méthode la plus utilisée en France étant la méthode MERISE.

B. Présentation de la méthode Merise :

Merise est une méthode de conception, de développement et de réalisation de projets informatiques. Le but de cette méthode est d'arriver à recevoir un système d'information. La méthode Merise est basée sur la séparation des données et des traitements à effectuer en plusieurs modèles conceptuels et physiques.

La séparation des données et des traitements assure une longévité au modèle. En effet, l'agencement des données n'a pas à être souvent remanié, tandis que les traitements le sont plus fréquemment. La méthode Merise date de 1978-1979, et fait suite à une consultation nationale lancée en 1977 par le ministère de l'industrie dont le but de choisir des sociétés de conseil en informatique afin de définir une méthode de conception de systèmes d'information. Les deux principales sociétés ayant mis au point cette méthode sont le CTI (Centre Technique d'Informatique) chargé de gérer le projet, et le CETE (Centre d'Etudes Technique de l'Equipements) implanté à Aix-en-Provence.

C. Coin technique de la conception

• 1 - La méthode Merise

Analyser un système d'information déroute parfois le non-initié, car traduire un environnement de travail en symboles cabalistiques n'est pas très habituel pour qui ne connaît pas. Pourtant, avec une once de théorie et deux grammes de pratique, on se rend compte que le processus est très abordable, soumis à quelques règles simples, faciles à acquérir et qui s'appliquent toujours de la même manière. La méthode décrite ici est Merise, elle est française et a plus de 20 ans. Elle consiste à recevoir un Modèle Conceptuel de Donnée (MCD), le transposer en Modèle Logique de Donnée Relationnelles (MLDR), puis à générer le Modèle Physique correspondant (MPD). C'est la plus répandu des technique d'analyse de bae de donnée. Nous étudierons particulièrement aujourd'hui la construction du Modèle Conceptuel de Donnée et de ses 5 caractéristiques :

- Entités ;
- Propriétés ;
- Identifiant ;
- Associations ;
- Cardinalités.

• 2- Système d'information.

La priorité est de transformer ce que l'on veut analyser en mots simples. L'écriture de cette petite rédaction permet à elle seule de bien comprendre ce que l'on va modéliser. Il s'agit à ce stade d'établir un lien entre

L'informaticien et les utilisateurs, il ne faut donc pas hésiter à faire relire votre petit texte et à poser toutes les questions qui vous viennent à l'esprit afin de bien analyser l'existant. La difficulté principale est d'arriver à faire abstraction de vos habitudes de programmation : à ce stade, nus

- **L'ETUDE de l'EXISTANT.**

L'étude de l'existant consiste à collecter l'ensemble des éléments qui permettent de décrire le système d'information (Document, données, circulation des flux....)

Cette étude est basée sur l'exploitation du système documentaire de l'école et sur la conduite d'entretien avec les intervenants et les personnels de l'école. Notre école est société S.A.R.L qu'elle a pour but former des jeunes licenciés, Bacheliers ou niveau bac.

1) Le dictionnaire de données

Le dictionnaire de données est l'ensemble des informations gérées par le système de gestion de base de données. Il permet de savoir quelles sont les données de base et de connaître leur structure.

Voici les règles de gestion de ce projet :

Champ	Type	Description	entité	Observations
id_client	INT	Identifiant unique de client	Client	Identifiant unique
nom_complet	VARCHAR	Nom et prénom de client		
telephone	VARCHAR	Téléphone de client		
email	VARCHAR	Email de client		Adresse électronique
adresse	TEXT	Adresse de client		Localisation
id_produit	NUMÉRIQUE	Identifiant unique de produit	Produit	Identifiant unique
code_barres	TEXTE	Code EAN ou interne		
designation	TEXTE	Nom du produit		
description	TEXTE	Détails techniques		
prix_unitaire	NUMÉRIQUE	Prix de vente de base		
seuil_alerte	NUMÉRIQUE	quantité minimum avant réapprovisionnement		
quantité	NUMÉRIQUE	Quantité de produit		
id_categorie	NUMÉRIQUE	Identifiant de Catégories		Identifiant unique

id_categorie	INT	Identifiant de la catégorie	Ctégorie	Identifiant unique
nom_categorie	VARCHAR	Nom de la catégorie		
id_entrepot	NUMÉRIQUE	Identifiant de l'entrepôt	Entropot	Identifiant unique
nom_magasin	TEXTE	Nom du site (ex: Dépôt Rabat)		
adresse	TEXTE	Adresse physique de l'entrepôt		
id_commande	NUMÉRIQUE	Identifiant du ligne de commande	Ligne_Achat	Identifiant unique
id_lot	NUMÉRIQUE	Identifiant de lot		Identifiant unique
id_produit	NUMÉRIQUE	identifiant de Produits		Identifiant unique
quantite	NUMÉRIQUE	quantité commandée		
Prix_achat	NUMÉRIQUE	Prix d'achat		
id_commande	NUMÉRIQUE	Numéro de commande	Commande_achat	Identifiant unique
id_utilisater	NUMÉRIQUE			Identifiant unique
date_commande	DATE	Date de création de commande		
statut	TEXTE	Statut de commande En attente, Reçue, Partielle		
id_fournisseur	NUMÉRIQUE	Idenifiant de Fournisseurs		Identifiant unique
id_vente	NUMÉRIQUE	Numéro de facture (identifiant)	Ligne_vente	Identifiant unique
id_produit	NUMÉRIQUE	identifiant de Produits		Identifiant unique
Id_lot	NUMÉRIQUE	identifiant d'utilisateur		Identifiant unique
Quantite	NUMÉRIQUE	quantité vendue		
prix_vente	NUMÉRIQUE	Prix au moment de la vente		

id_vente	NUMÉRIQUE	Numéro de facture (identifiant)	Vente	Identifiant unique
date_vente	DATE	Date de transaction		datetime
id_utilisateur		Identifiant de L'utilisateur		Identifiant unique
Id_bon_livraison	NUMÉRIQUE	Identifiant de bon livraison		Identifiant unique
id_client	NUMÉRIQUE	Identifiant de Client		Identifiant unique
id_fournisseur	INT	Identifiant de fournisseur	Fournisseur	Identifiant unique
nom_complet	VARCHAR	Nom de fournisseur		
telephone	VARCHAR	Téléphone de fournisseur		
email	VARCHAR	Email de fournisseur		
adresse	TEXT	Adresse de fournisseur		
id_produit	NUMÉRIQUE	Identifiant de produit	Ligne_stock	Identifiant unique
id_emplacement	NUMÉRIQUE	Identifiant unique de L'emplacement		Identifiant unique
Quantite	NUMÉRIQUE	Quantité stocker		Integer
id_emplacement	NUMÉRIQUE	Identifiant d'emplacement	Emplacement	Identifiant unique
code_emplacement	TEXTE	Code du rayon		
id_entrepot	NUMÉRIQUE	Identifiant d'Entrepots		Identifiant unique
id_lot	NUMÉRIQUE	Identifiant de Lot	Lot	Identifiant unique
date_fabrication	Date	Date de fabrication		Datetime
date_peremption	Date	Date de peremption		Datetime
id_produit	NUMÉRIQUE	Identifiant unique de produit		Identifiant unique
id_invenaire	NUMÉRIQUE	Identifiant de L'inventaire	inventaire	Identifiant unique
date_invenaire	Date	Date de L'inventaire		Datetime
id_entrepot	NUMÉRIQUE	Identifiant d'emplacement		Identifiant unique

id_utilisateur	NUMÉRIQUE	Identifiant de l'utilisateur		
id_produit	NUMÉRIQUE	Identifiant unique de produit		Identifiant unique
rmarque	TEXTE	Quantite réelle en stock		
id_bon_Livraison	NUMÉRIQUE	Identifiant de la Livraison	Bon_Livraison	Identifiant unique
date_Livraison	Date	Date de La livraison		Datetime
adresse	TEXTE	Adresse de La livraison		Localisation
id_utilisateur	NUMÉRIQUE	Identifiant de l'utilisateur	Utilisateur	Identifiant unique
id_role	NUMÉRIQUE	Identifiant de		Identifiant unique
nom_utilisateur	TEXTE	Nom d'utilisateur		
mot_passe	TEXTE	Mot de passe		
is_active	DEFAULT(0,1)	Compte D'utilisateur activer ou non		Default
id_role	NUMÉRIQUE	Identifiant de role	Role	Identifiant unique
libelle	TEXTE	Nom de role		
id_audit	Auto increment	Identifiant d'Audit	AUDIT	Identifiant unique
id_cible	NUMÉRIQUE	Identifiant de table cible		Identifiant unique
nom_table		Nom de table		
type_op	DEFAULT (insert,update,delete)	Type D'opoperation (Insert,Update,Delete)		DEFAULT
date_op	DATE	Date D'opoperation		
nom_utilisateur	TEXTE	Nom D'utilisateur		
ancienne_valeur	TEXTE	Champs supprimer ou modifier		
nouvelle_valeur	TEXTE	Champs Ajouter ou modifier		
id_utilisateur	NUMÉRIQUE	Identifiant de l'utilisateur		Identifiant unique
id_utilisateur_connecte	NUMÉRIQUE	Identifiant de l'utilisateur		Identifiant unique

		Connecter		
id_paiement	NUMÉRIQUE	Identifiant de paiement	Paiement	Identifiant unique
date_paiement		Date de paiement		DATETIME
mode_paiement	DEFAULT (espèces,chèque,cache ...)	Mode de paiement		DEFAULT
id_vente	NUMÉRIQUE	Identifiant de vente		Identifiant unique
id_produit	NUMÉRIQUE	Identifiant de produit	Ligne_inventaire	Identifiant unique
id_inventaire	NUMÉRIQUE	Identifiant de l'inventaire		Identifiant unique
stock_theorique	NUMÉRIQUE	Stocke en théorie		
stock_reel	NUMÉRIQUE	Stock en réalité		
id_produit	NUMÉRIQUE	Identifiant de produit	Fournir	Identifiant unique
id_fournisseur	NUMÉRIQUE	Identifiant de fournisseur		Identifiant unique
quantite	NUMÉRIQUE	Quantité fournit		integer

2) Règles de Gestion :

Les règles de gestion sont des règles qui définissent comment les données doivent être traitées et mises à jour dans le système d'information.

Voici les règles de gestion de ce projet :

- RG1 : Un client peut effectuer 0 à n ventes
- RG2 : Une vente est effectuée par un et un seul client
 - RG3 : Une vente contient au minimum 1 produit
 - RG4 : Un produit peut apparaître dans 0 à n ventes
- RG5 : Une vente peut avoir 0 ou 1 bon de livraison
- RG6 : Un bon de livraison correspond à une seule vente
- RG7 : Un produit appartient à une et une seule catégorie
 - RG8 : Une catégorie contient 0 à n produits

- RG9 : Un fournisseur peut fournir 0 à n produits
 - RG10 : Un produit peut être fourni par 0 à n fournisseurs
 - RG11 : Un fournisseur peut recevoir 0 à n commandes d'achat
 - RG12 : Une commande d'achat est passée auprès d'un et un seul fournisseur
 - RG13 : Une commande d'achat contient au minimum 1 produit
 - RG14 : Un produit peut apparaître dans 0 à n commandes d'achat
 - RG15 : Un produit peut avoir 0 à n lots
 - RG16 : Un lot appartient à un et un seul produit
 - RG17 : Un produit peut être stocké dans 0 à n emplacements
 - RG18 : Un emplacement peut contenir 0 à n produits
 -
 - RG19 : Un emplacement appartient à un et un seul entrepôt
 - RG20 : Un entrepôt contient 0 à n emplacements
 - RG21 : Une vente peut être réglée par 0 à n paiements
 - RG22 : Un paiement concerne une et une seule vente
 - RG23 : Un inventaire concerne 1 à n produits
 - RG24 : Un produit peut apparaître dans 0 à n inventaires
 - RG25 : Un utilisateur possède un seul rôle.
- Les rôles sont : (Administrateur, Magasinier, Responsable)
- RG26 : Chaque action (Insertion, Modification, Suppression...) doit être tracée.
 - RG27 : L'utilisateur de chaque action doit être identifier.
 - RG28 : La date et l'heure précis de L'évènement doit être enregistrer
 - RG29 : Pour une modification, l'ancienne valeur et la nouvelle valeur doit être conserver.

3) Le Modèle Conceptuel des Données (M.C.D) :

Le Modèle conceptuel des Données (MCD) est un modèle qui représente les données d'un système d'information de manière conceptuelle, il permet de représenter les entités, les attributs et les relations entre eux.

Voici le modèle conceptuel des données de ce projet :

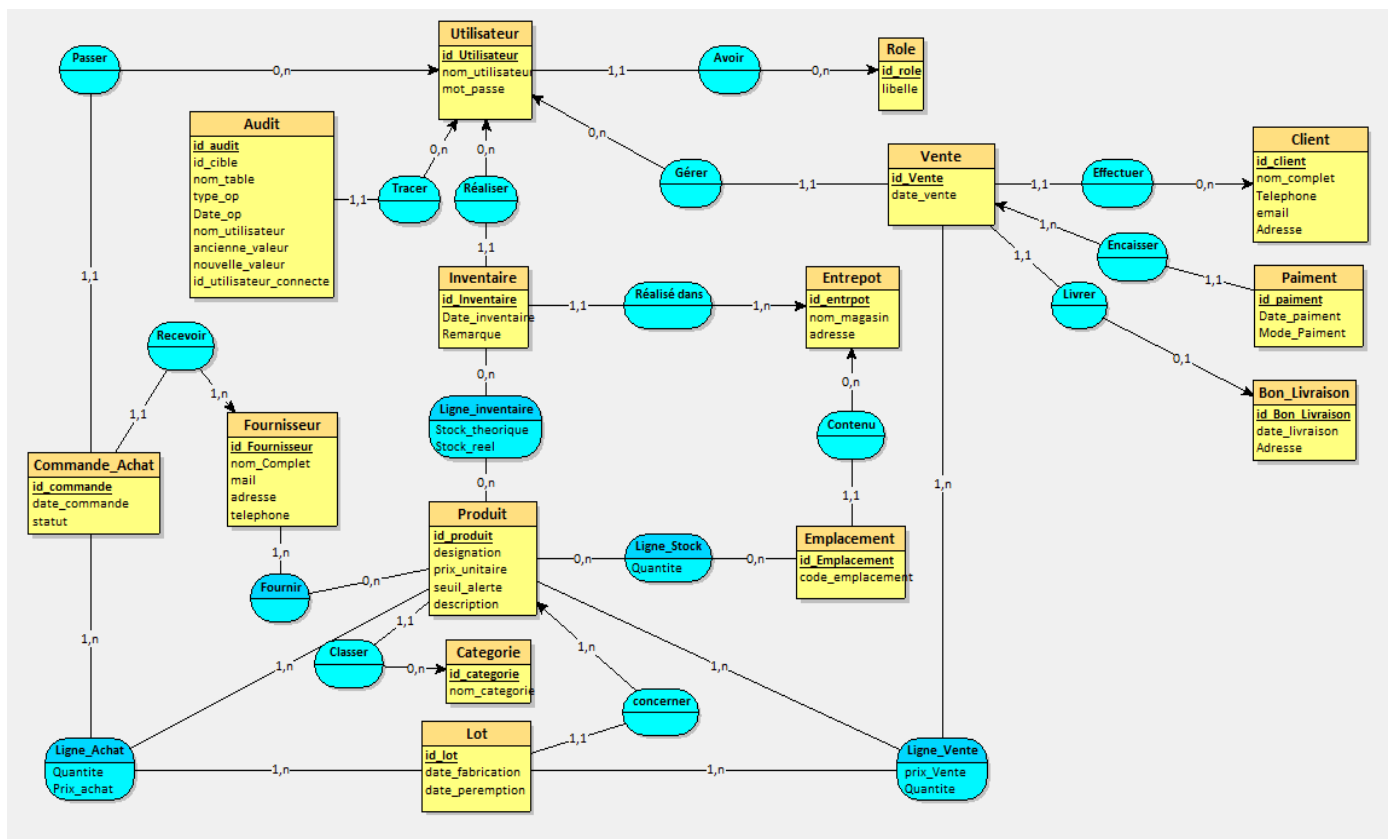


Figure 1 : modèle conceptuel de données

4) Le Modèle logique des Données (M.L.D) :

Le modèle logique des données est une représentation du modèle conceptuel en terme d'organisation de données, il traduit les données automatisées et leurs liens dans un formalisme plus proche du langage de l'ordinateur.

Voici le modèle logique des données de ce projet :

- Client (#id_Client, nom_client, Telephone, email, Adresse);
- Bon Livraison (#id_Bon_Livraison, date_livraison , Adresse);
- Fournisseur (#id_Fournisseur , nom_fournisseur, mail , adresse , telephone);
- Categorie (#id_Categorie, nom_categorie);
- Entrepot (#id_Entrepot , nom_magasin , adresse);
- Role (#id_role , libelle);
- Produit (#id_produit, #id_categorie* , designation , prix_unitaire , seuil_alerte , description);
- Emplacement (#id_Emplacement , #id_entrepot* , code_emplacement);
- Lot (#id_lot , #id_produit* , date_fabrication , date_peremption);
- Utilisateur (#id_Utilisateur, #id_role* , Login , mot_passe);
- Inventaire (#id_Inventaire, #id_entrepot* , #id_Utilisateur* , Date_inventaire, Remarque);
- Audit (#id_Audit, #id_Utilisateur* , #id_Enregistrement* , #id_Utilisateur* , type_op, ancienne_valeur , nouvelle_valeur, nom_table);
- Vente (#id_Vente , date_vente , #id_Utilisateur* , #id_Bon_Livraison* , #id_client*);
- Commande_Achat (#id_commande , date_commande , statut , #id_Utilisateur* , #id_Fournisseur*);
- Paiement (#id_paiement, Date_paiement, Mode_Paiement, #id_Vente*);
- Fournir (#id_produit* , #id_Fournisseur*);
- Ligne_Stock (#id_produit* , #id_Emplacement* , Quantite);
- Ligne_Achat (#id_produit* , #id_commande* , #id_lot* , Prix_achat , Quantite);
- Ligne_Vente (#id_Vente* , #id_produit* , #id_lot* , Quantite , prix_Vente);
- Ligne_inventaire (#id_produit* , #id_Inventaire* , Stock_theorique , Stock_reel);

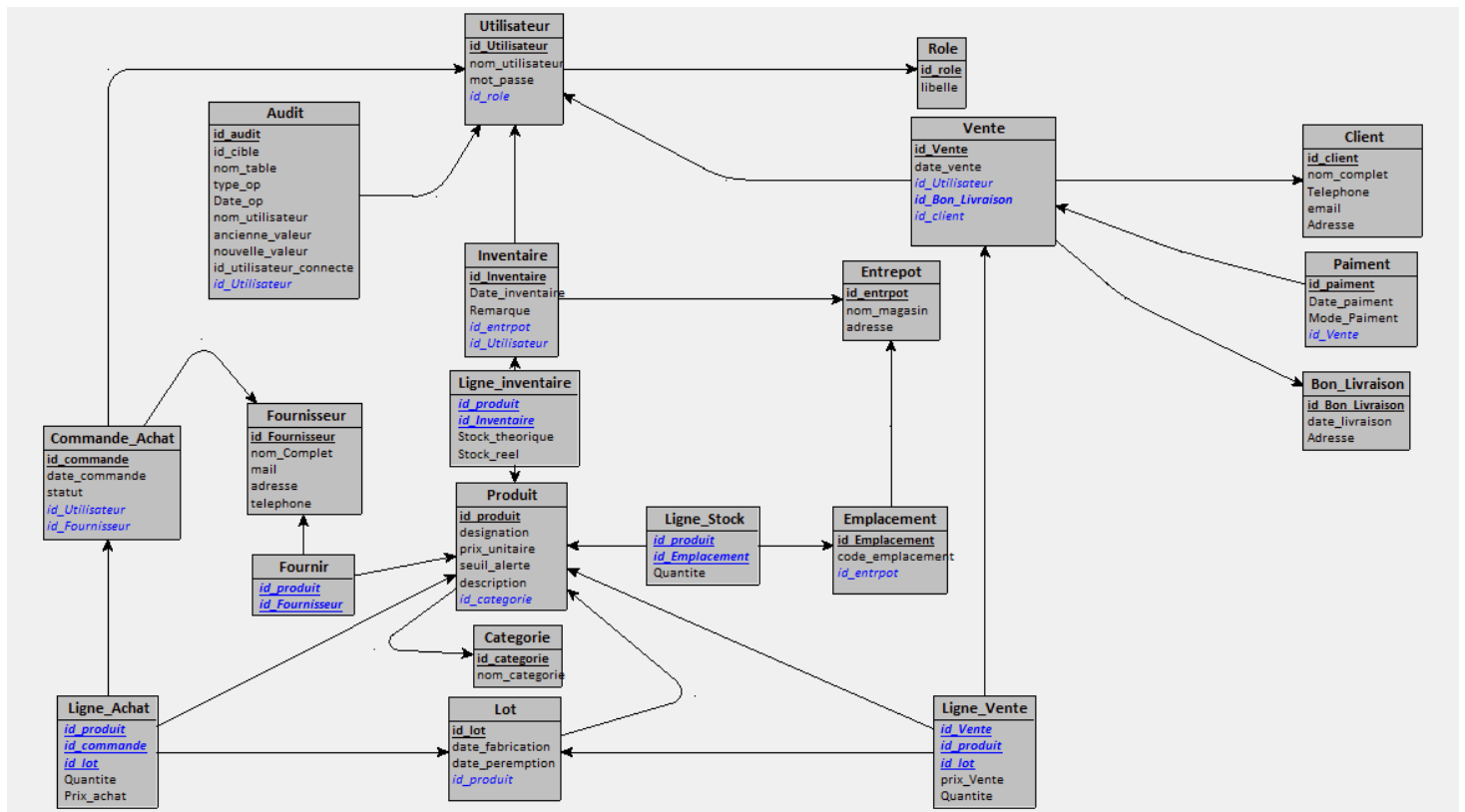
[*] : Clé étrangère

[#] : Clé primaire

5) Le Modèle physique des Données (M.P.D) :

Le Modèle Physique des Données (MPD) est une Représentation concrète de la structure de la base de données, définissant les caractéristiques techniques de la base de données telles que la taille des champs et les types de données, en fonction du système de gestion de base de données.

Voici le modèle physique des données de ce projet :

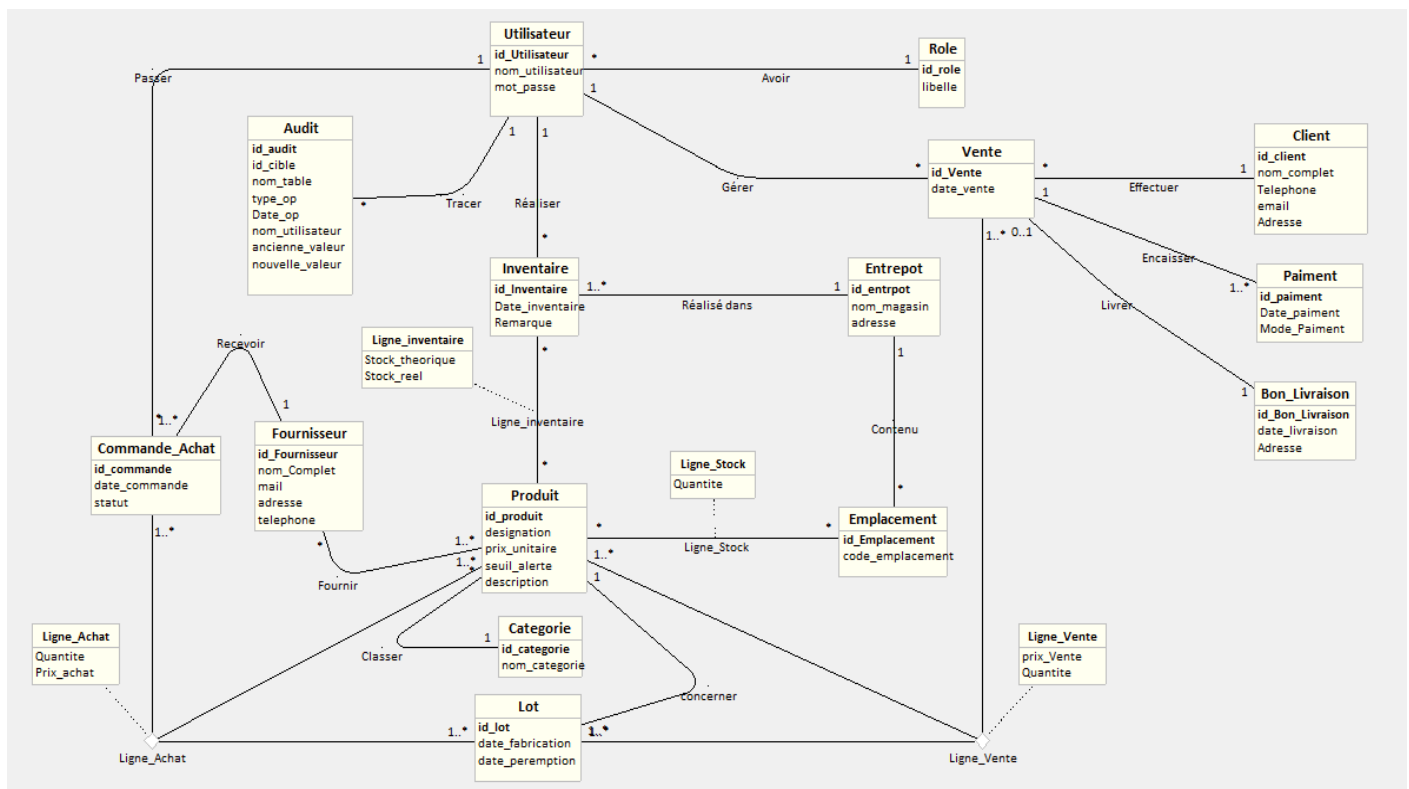


D. Présentation de la méthode UML :

Le langage UML (Unified Modeling Language) :, ou Langage de Modélisation Unifié, est un standard international utilisé en génie logiciel pour formaliser, visualiser et documenter la conception d'un système informatique. Contrairement à ce que l'on pourrait penser, UML n'est pas une méthode de programmation ni un langage de code, mais plutôt une boîte à outils visuelle. Il permet de traduire des besoins fonctionnels complexes en schémas standardisés, facilitant ainsi la communication entre les développeurs, les chefs de projet et les clients. UML s'articule autour de deux grandes catégories de diagrammes : les diagrammes structurels (qui décrivent l'architecture statique du système) et les diagrammes comportementaux (qui décrivent la dynamique et les interactions).

2) Le Diagramme De Classe :

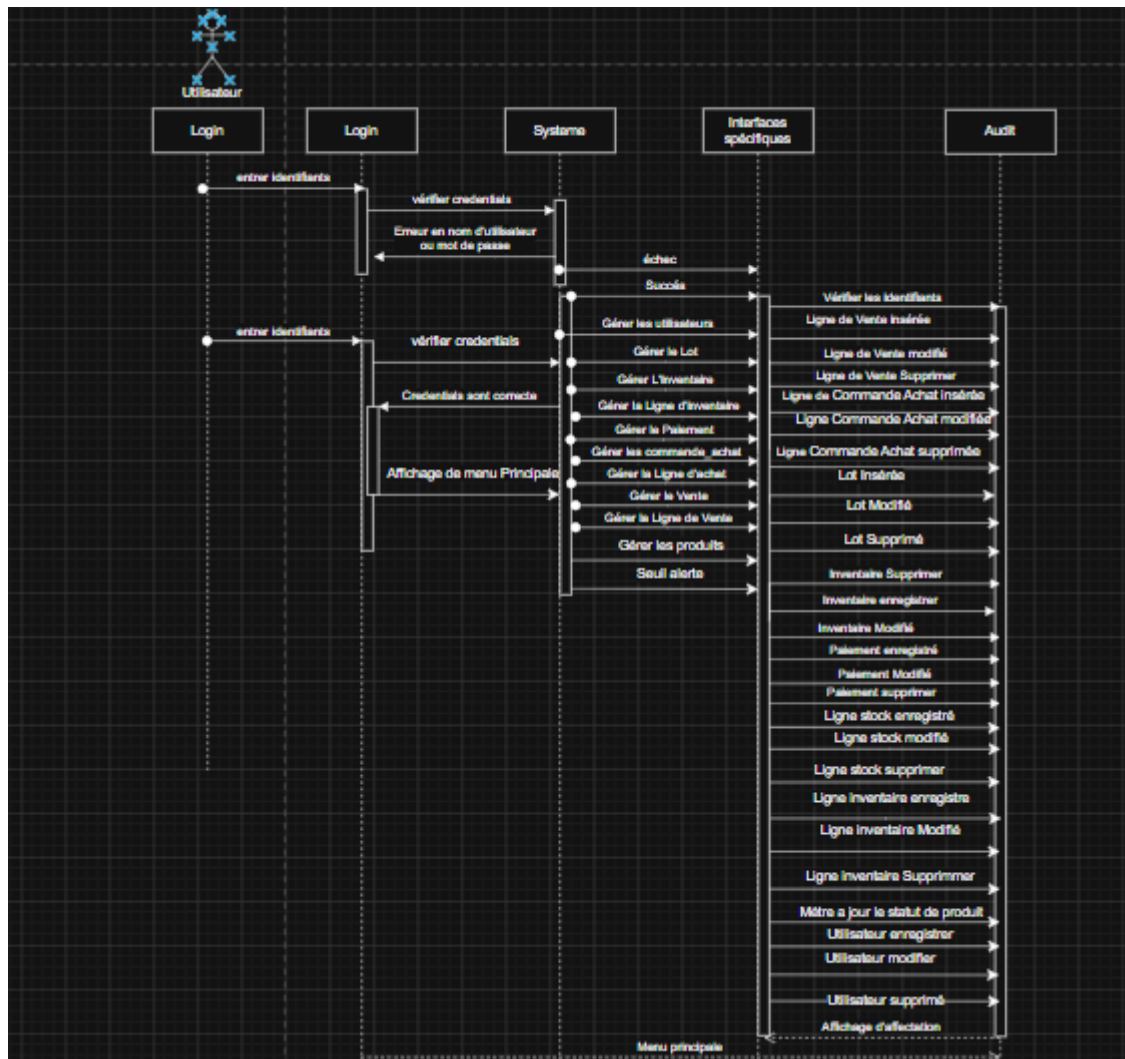
Le Diagramme de Classe (Class Diagram) est le cœur de la modélisation structurelle en UML. Il représente la structure statique du système en mettant en évidence les données et les concepts clés du domaine de l'application. Ce diagramme modélise les classes (qui regroupent les attributs/données et les méthodes/fonctions) ainsi que les relations qui les unissent (comme l'association, l'héritage, l'agrégation ou la composition). Chaque relation est précisée par des multiplicités (par exemple 1..* ou 0..1) qui dictent les règles de gestion des données. Ce diagramme est crucial car il sert de pont direct entre la conception théorique et la génération du code (en programmation orientée objet) ou la création de la base de données.



3) Le Diagramme De Séquence :

Le Diagramme de Séquence (Sequence Diagram) est un diagramme dynamique qui montre comment les différents objets du système interagissent entre eux au fil du temps.

Il est généralement utilisé pour détailler le scénario technique d'un cas d'utilisation précis. Chronologique, il se lit de haut en bas et met en scène les objets (représentés par des "lignes de vie" verticales) qui s'échangent des messages (représentés par des flèches horizontales). Il permet de visualiser précisément l'ordre d'exécution des fonctions, les appels de méthodes, les retours d'information, ainsi que la création ou la destruction d'objets. C'est l'outil de référence pour les développeurs pour comprendre la logique d'exécution du code.



Technologie :

Langage de programmation : C#

Framework : .NET Framework 4.8

Interface graphique : Windows Forms (WinForms)

Environnement de développement (IDE) : Microsoft Visual Studio Community 2019

Système de Gestion de Base de Données (SGBD) : Microsoft SQL Server

Outil d'administration de la base de données : SQL Server Management Studio (SSMS)

Accès aux données : ADO.NET

- SqlConnection
- SqlCommand
- SqlDataAdapter

Procédures stockées :

- Audit automatique des lignes (lot, achat, vente, stock, lot, inventaire et son ligne)
- Validation et traitement des données

Triggers :

- Audit automatique des opérations de table Utilisateur et Status en table Produit (INSERT, UPDATE, DELETE)
- Contrôle du seuil d'alerte des produits

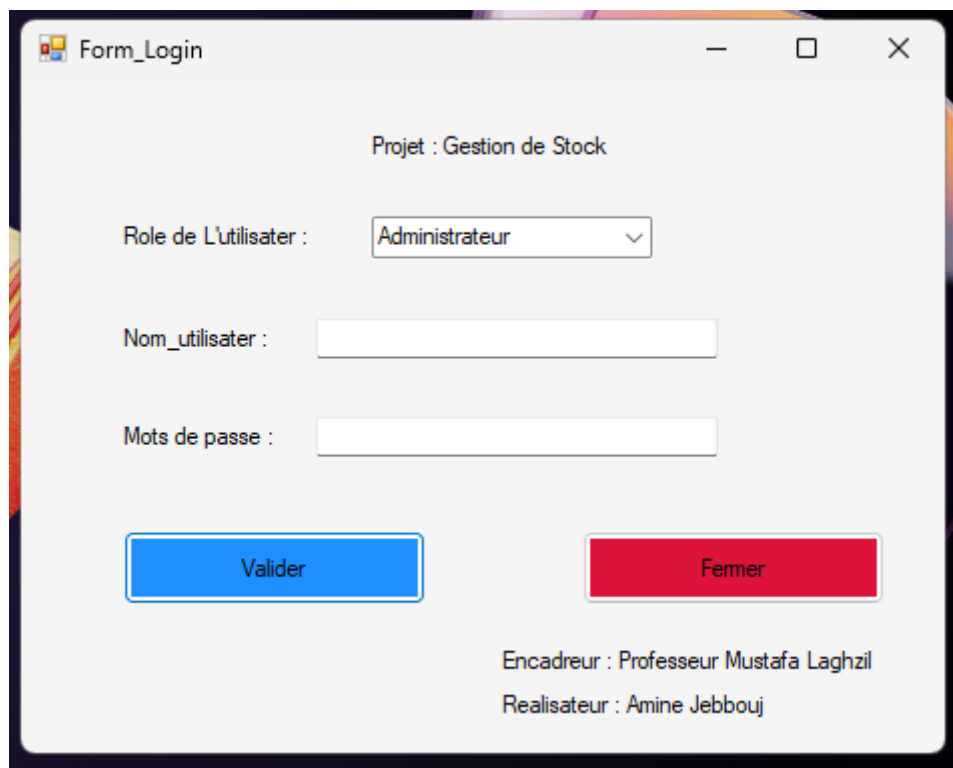
Démonstration :

I. Partie application :

Interfaces de l'application :

A.

Interface Authentification Description : Cette interface permet à l'utilisateur de saisir ses identifiants (nom d'utilisateur et mot de passe). Le système vérifie les informations saisies. En cas d'erreur, un message d'alerte est affiché et l'accès est refusé. Si les identifiants sont corrects, l'utilisateur est redirigé vers le menu principal.



Form_Login

Projet : Gestion de Stock

Role de L'utilisateur : Administrateur

Nom_utilisateur :

Mots de passe :

Valider Fermer

Encadreur : Professeur Mustafa Laghzil
Realisateur : Amine Jebbouj

- **Code de bouton fermer :**

```
private void button2_Click(object sender, EventArgs e)
{
    this.Close();
}
```

• Code de bouton Valider

```
private void button1_Click(object sender, EventArgs e)
{
    using (SqlConnection cnx = new SqlConnection(conn))
    {
        string Q = "SELECT id_utilisateur FROM Utilisateur WHERE nom_utilisateur = @u AND
mot_passe = @mdp AND id_role = @r AND is_active = '1' ";

        SqlCommand cmd = new SqlCommand(Q, cnx);
        cmd.Parameters.Add("@u", SqlDbType.VarChar, 50).Value = nu.Text.Trim();
        cmd.Parameters.Add("@mdp", SqlDbType.VarChar, 50).Value = mp.Text.Trim();
        cmd.Parameters.Add("@r", SqlDbType.Int).Value = Convert.ToInt32(R.SelectedValue);

        try
        {
            cnx.Open();
            using (SqlDataReader dr = cmd.ExecuteReader())
            {
                if (dr.Read())
                {
                    SecurityContext.IdUtilisateur = Convert.ToInt32(dr["id_utilisateur"]);
                    SecurityContext.RoleConnecte = R.Text;
                    new Menu_Principale().Show();
                    this.Hide();
                }
                else
                {
                    MessageBox.Show
                    ("NOM D'UTILISATEUR OU MOT DE PASSE OU ROLE EST INCORRECT !!");
                }
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message);
        }
    }
}
```


B. Interface Tableau de bord.

Description :

Le tableau de bord constitue le point d'entrée principal de l'application. Il offre un accès aux différentes fonctionnalités de gestion : utilisateurs, lots, inventaires, ventes, commandes d'achat, paiements et suivi des seuils de produits. Chaque module est accessible via un menu clair et structuré.

Code de bouton Utilisateur



Exemple de codes des boutons De Menu :

```
private void button13_Click(object sender, EventArgs e)
{
    new Utilisateur().Show();
}
```

Code de bouton Lot

```
private void button13_Click(object sender, EventArgs e)
{
    new Lot().Show();
}
```

C. Interfaces spécifiques Utilisateur

Description :

Cette interface permet de gérer les utilisateurs du système (ajout, modification, suppression, activation/désactivation).

	id_utilisateur	nom_utilisateur	mot_passe	id_role	is_active
▶	1	amine	1234	1	
	2	anass	4321	3	
	4	mohamed	4321	1	
	12	anas	1234	2	
	14	mohamed	1234	1	
*					

Cette table possède des triggers d'audit (INSERT, UPDATE, DELETE) qui assurent la traçabilité des opérations effectuées sur les utilisateurs.

Exemples de crud pour les tables qui ont des procedures

1. code de bouton Nouveau

```
private void button1_Click(object sender, EventArgs e)
```

```
{    idu.Clear();
```

```
nu.Clear();
```

```
mp.Clear();
```

```
idu.Select(); }
```

2. code de bouton Désactiver

```
private void button8_Click(object sender, EventArgs e)
```

```
{    using (SqlConnection db = new SqlConnection(Chemin))
```

```
{    db.Open();
```

```
SqlCommand cast = new SqlCommand("SET CONTEXT_INFO @id_u;", db);
```

```
cast.Parameters.Add("@id_u", SqlDbType.VarBinary, 128).Value =  
SecurityContext.IdBytes;
```

```
cast.ExecuteNonQuery();
```

```
string checkQ = "SELECT is_active FROM Utilisateur WHERE id_utilisateur =  
@id";
```

```

SqlCommand checkCmd = new SqlCommand(checkQ, db);

checkCmd.Parameters.Add("@id", SqlDbType.Int).Value= int.Parse(idu.Text.Trim());

int currentState = Convert.ToInt32(checkCmd.ExecuteScalar());

int newState = (currentState == 1) ? 0 : 1;

string Q = "UPDATE Utilisateur SET is_active = @newState WHERE id_utilisateur = @id";

SqlCommand Cmd = new SqlCommand(Q, db);

Cmd.Parameters.Add("@newState", SqlDbType.Int).Value = newState;

Cmd.Parameters.Add("@id", SqlDbType.Int).Value = int.Parse(idu.Text.Trim());

Cmd.ExecuteNonQuery();

Utilisateur_Load(sender, e);

if (newState == 0)

    MessageBox.Show("Utilisateur désactivé !");

else

    MessageBox.Show("Utilisateur activé !"); } }

```

3. code de bouton Actualiser

```
private void button2_Click(object sender, EventArgs e)

{
    using (SqlConnection db = new SqlConnection(Chemin))

    {

        db.Open();

        string actualiser = "SELECT id_utilisateur, nom_utilisateur, mot_passe, id_role
        FROM Utilisateur WHERE is_active = 1";

        SqlCommand Cmd = new SqlCommand(actualiser, db);

        SqlDataReader Fiche = Cmd.ExecuteReader();

        dataGridView1.Rows.Clear();

        while (Fiche.Read()) {

            dataGridView1.Rows.Add(

                Fiche["id_utilisateur"],

                Fiche["nom_utilisateur"],

                Fiche["mot_passe"],

                Fiche["id_role"]);    }

        Fiche.Close(); }    }
```

4. code de bouton Enregistrer

```
private void button7_Click(object sender, EventArgs e)

{
    using (SqlConnection db = new SqlConnection(Chemin))

{
    db.Open();

SqlCommand cast = new SqlCommand("SET CONTEXT_INFO @id_u;", db);

cast.Parameters.Add("@id_u", SqlDbType.VarBinary, 128).Value =
SecurityContext.IdBytes;

cast.ExecuteNonQuery();

string Cmd = "insert into Utilisateur

(id_utilisateur,nom_utilisateur,mot_passe,id_role) values (@id, @n, @mdp, @ir) ";

SqlCommand insrt = new SqlCommand(Cmd, db);

insrt.Parameters.AddWithValue("@id", idu.Text.Trim());

insrt.Parameters.AddWithValue("@n", nu.Text.Trim());

insrt.Parameters.AddWithValue("@mdp", mp.Text.Trim());

insrt.Parameters.Add("@ir",

SqlDbType.Int).Value = Convert.ToInt32(R.Selected.Value);

insrt.ExecuteNonQuery();

Utilisateur_Load(sender, e);
```

```
MessageBox.Show(" Utilisateur Bien Ajouter !! .");
```

```
}}
```

5. code de bouton Rechercher

```
private void button3_Click(object sender, EventArgs e)
```

```
{    using (SqlConnection db = new SqlConnection(Chemin))
```

```
{    db.Open();
```

```
string Q = "SELECT id_utilisateur, nom_utilisateur, mot_passe, id_role, is_active  
FROM Utilisateur WHERE id_utilisateur = @id";
```

```
SqlCommand Cmd = new SqlCommand(Q, db);
```

```
Cmd.Parameters.AddWithValue("@id", idu.Text.Trim());
```

```
SqlDataReader Fiche = Cmd.ExecuteReader();
```

```
dataGridView1.Rows.Clear();
```

```
while (Fiche.Read())
```

```
{    dataGridView1.Rows.Add(
```

```
Fiche["id_utilisateur"],
```

```
Fiche["nom_utilisateur"],
```

```
Fiche["mot_passe"],
```

```
Fiche["id_role"],
```

```
(Convert.ToInt32(Fiche["is_active"]) == 1 ? "Actif" : "Désactivé")    );}
```

```
Fiche.Close();    }
```

6. code de bouton Modifier

```
using (SqlConnection db = new SqlConnection(Chemin))
```

```
{    db.Open();
```

```
SqlCommand cast = new SqlCommand("SET CONTEXT_INFO @id_u;", db);
```

```
cast.Parameters.Add("@id_u", SqlDbType.VarBinary, 128).Value =  
SecurityContext.IdBytes;
```

```
cast.ExecuteNonQuery();
```

```
if (string.IsNullOrEmpty(idu.Text) ||
```

```
string.IsNullOrEmpty(nu.Text) ||
```

```
string.IsNullOrEmpty(mp.Text) ||
```

```
R.SelectedIndex == -1)
```

```
{    MessageBox.Show("Veuillez remplir tous les champs obligatoires !");
```

```
return;    }
```

```
int idUtilisateur, idRole;
```

```
if (!int.TryParse(idu.Text.Trim(), out idUtilisateur))
```

```
{
```



```
MessageBox.Show("ID utilisateur invalide !");
```

```
return;    }
```

```
idRole = Convert.ToInt32(R.SelectedValue);
```

```
string Q = "UPDATE Utilisateur SET nom_utilisateur = @n, mot_passe = @mdp,  
id_role = @ir WHERE id_utilisateur = @id";
```

```
SqlCommand updt = new SqlCommand(Q, db);
```

```
updt.Parameters.Add("@id", SqlDbType.Int).Value = idUtilisateur;
```

```
updt.Parameters.Add("@n", SqlDbType.VarChar, 50).Value = nu.Text.Trim();
```

```
updt.Parameters.Add("@mdp", SqlDbType.VarChar, 50).Value = mp.Text.Trim();
```

```
updt.Parameters.Add("@ir", SqlDbType.Int).Value = idRole;
```

```
updt.ExecuteNonQuery();
```

```
Utilisateur_Load(sender, e);    }
```

```
MessageBox.Show("Modification bien effectuée !");    }
```

7. code de bouton Supprimer

```
using (SqlConnection db = new SqlConnection(Chemin))
```

```
{    db.Open();
```

```
SqlCommand cast = new SqlCommand("SET CONTEXT_INFO @id_u;", db);
```

```
cast.Parameters.Add("@id_u", SqlDbType.VarBinary, 128).Value =
```

```
SecurityContext.IdBytes;
```

```
cast.ExecuteNonQuery();
```

```
if (string.IsNullOrEmpty(idu.Text) ||
```

```
string.IsNullOrEmpty(nu.Text) ||
```

```
string.IsNullOrEmpty(mp.Text) ||
```

```
R.SelectedIndex == -1)
```

```
{    MessageBox.Show("Veuillez remplir tous les champs obligatoires !");
```

```
return;    }
```

```
int idUtilisateur, idRole;
```

```
if (!int.TryParse(idu.Text.Trim(), out idUtilisateur))
```

```
{    MessageBox.Show("ID utilisateur invalide !");
```

```
return;    }
```

```
idRole = Convert.ToInt32(R.SelectedValue);
```

```
string Q = "UPDATE Utilisateur SET nom_utilisateur = @n, mot_passe = @mdp,  
id_role = @ir WHERE id_utilisateur = @id";
```

```
SqlCommand updt = new SqlCommand(Q, db);
```

```
updt.Parameters.Add("@id", SqlDbType.Int).Value = idUtilisateur;
```

```

updt.Parameters.Add("@n", SqlDbType.VarChar, 50).Value = nu.Text.Trim();

updt.Parameters.Add("@mdp", SqlDbType.VarChar, 50).Value = mp.Text.Trim();

updt.Parameters.Add("@ir", SqlDbType.Int).Value = idRole;

updt.ExecuteNonQuery();

Utilisateur_Load(sender, e);}

MessageBox.Show(" Utilisateur Bien supprimé !");    }

```

code de bouton Fermer

```

private void button6_Click(object sender, EventArgs e)

{
    this.Close();
}

```

➤ Code de Trrigger Utilisateur Insertion :

```

CREATE TRIGGER trg_Audit_utilisateur_insert_Login

ON [dbo].[Utilisateur]

AFTER insert

AS

BEGIN

SET NOCOUNT ON;

DECLARE @id_u INT;

```

```
SELECT @id_u = CAST(CONTEXT_INFO() AS INT);
```

```
INSERT INTO Audit
```

```
(    id_cible,
```

```
nom_table,
```

```
type_op,
```

```
Date_op,
```

```
id_utilisateur,
```

```
[id_utilisateur_connecte],
```

```
nom_utilisateur,
```

```
ancienne_valeur,
```

```
nouvelle_valeur )
```

```
SELECT
```

```
i.id_utilisateur,
```

```
'Utilisateur',
```

```
'Insertion',
```

```
GETDATE(),
```

```
i.id_utilisateur,
```

```

        @id_u,

        i.nom_utilisateur,

        NULL,

        'nom_utilisateur :' + ISNULL(CAST(i.nom_utilisateur AS NVARCHAR(80)), '') +

        ' mot_passe :' + ISNULL(CAST(i.mot_passe AS NVARCHAR(80)), '') +

        ' id_role :' + ISNULL(CAST(i.id_role AS NVARCHAR(80)), '')

FROM inserted i;  END

```

➤ Code de Trrigger Utilisateur Modification :

```

CREATE TRIGGER trg_Audit_utilisateur_update_Login

ON    [dbo].[Utilisateur]

AFTER UPDATE

AS

BEGIN

SET NOCOUNT ON;

DECLARE @id_u INT;

SELECT @id_u = CAST(CONTEXT_INFO() AS INT);

INSERT INTO Audit

```

```
( id_cible,  
  
nom_table,  
  
type_op,  
  
Date_op,  
  
id_utilisateur,  
  
id_utilisateur_connecte,  
  
nom_utilisateur,  
  
ancienne_valeur,  
  
nouvelle_valeur )
```

```
SELECT
```

```
i.id_utilisateur,
```

```
'Utilisateur',
```

```
'Modification',
```

```
GETDATE(),
```

```
i.id_utilisateur,
```

```
@id_u,
```

```
i.nom_utilisateur,
```

```
'nom_utilisateur :' + ISNULL(CAST(d.nom_utilisateur AS NVARCHAR(80)), '') +  
mot_passe :' + ISNULL(CAST(d.mot_passe AS NVARCHAR(80)), '') + ' id_role
```

```
le :' + ISNULL(CAST(d.id_role AS NVARCHAR(80)), ''), 'nom_utilisateur :' +  
ISNULL(CAST(i.nom_utilisateur AS NVARCHAR(80)), '') + ' mot_passe :' +  
ISNULL(CAST(i.mot_passe AS NVARCHAR(80)), '') + ' id_role :' +  
ISNULL(CAST(i.id_role
```

```
AS NVARCHAR(80)), '') FROM inserted i
```

```
INNER JOIN deleted d ON i.id_utilisateur = d.id_utilisateur; END;
```



Code de Trigger Utilisateur Suppression :

```
CREATE TRIGGER trg_Audit_utilisateur_softdelete
```

```
ON [dbo].[Utilisateur]
```

```
AFTER UPDATE
```

```
AS
```

```
BEGIN
```

```
SET NOCOUNT ON;
```

```
DECLARE @id_u INT;
```

```
SELECT @id_u = CAST(CONTEXT_INFO() AS INT);
```

```
INSERT INTO Audit (
```

```
id_cible, nom_table, type_op, Date_op,
```

id_utilisateur, id_utilisateur_connecte, nom_utilisateur,

ancienne_valeur, nouvelle_valeur)

SELECT d.id_utilisateur, 'Utilisateur', 'Suppression (soft)', GETDATE(),

d.id_utilisateur, @id_u, d.nom_utilisateur,

'nom_utilisateur :' + ISNULL(CAST(d.nom_utilisateur AS NVARCHAR(80)), ") +

' mot_passe :' + ISNULL(CAST(d.mot_passe AS NVARCHAR(80)), ") +

' id_role :' + ISNULL(CAST(d.id_role AS NVARCHAR(80)), ") +

' is_active :' + ISNULL(CAST(d.is_active AS NVARCHAR(10)), "),

'is_active :' + ISNULL(CAST(i.is_active AS NVARCHAR(10)), ")

FROM deleted d

JOIN inserted i ON d.id_utilisateur = i.id_utilisateur

WHERE d.is_active = 1 AND i.is_active = 0;

END;

D. Interface Gestions des Produits

Description :

Description: Cette table permet de gérer les produits disponibles dans le stock, avec leurs informations principales (désignation, description, prix unitaire, seuil d'alerte, quantité, catégorie).

Produit

ID Produit :

ID Catégorie :

Désignation :

Prix Unitaire :

Description :

Quantité :

Nouveau

Actualiser

Enregistrer

Rechercher

Modifier

Supprimer

Fermer

	id_produit	designation	prix_unitaire	seuil_alerte	description_p	id_categorie	quantité
*							

Elle assure le suivi des articles sans nécessiter de procédure ou de trigger spécifique.

E. Interface Lots

Description :

Cette interface permet de gérer les lots de produits (date de fabrication, date de péremption).

The interface is a window titled "Lot" with standard Windows window controls (minimize, maximize, close). It contains the following elements:

- Input Fields:**
 - ID_Lot :** A text input field.
 - ID_Produit :** A text input field.
 - Date_Fabrication :** A date and time picker showing "01/07/2026 09:01".
 - Date_Pereption :** A date and time picker showing "01/07/2026 09:01".
- Table:** A table with 4 columns: `id_lot`, `date_fabrication`, `date_pereption`, and `id_produit`. The first row contains an asterisk (*) in the `id_lot` column. The rest of the table is greyed out.
- Action Buttons:** A vertical stack of buttons on the right side:
 - Nouveau** (Green)
 - Actualiser** (Orange)
 - Enregistrer** (Blue)
 - Rechercher** (Purple)
 - Modifier** (Blue)
 - Supprimer** (Red)
 - Fermer** (Grey)

Cette table est liée à une procédure stockée d'audit qui trace les insertions et suppressions , modifications.

Exemple de crud pour les Tables qui ont des procédures Stockées :

1. Code de bouton nouveau

```
private void button1_Click(object sender, EventArgs e)

{
    IDL.Clear();

    IDP.Clear();

    dateTimePicker1.Format = DateTimePickerFormat.Custom;

    dateTimePicker1.CustomFormat = "dd/MM/yyyy HH:mm";

    dateTimePicker2.Format = DateTimePickerFormat.Custom;

    dateTimePicker2.CustomFormat = "dd/MM/yyyy HH:mm";

    IDP.Select();
}
```

2. Code de bouton Actualiser

```
private void button2_Click(object sender, EventArgs e)

{
    using (SqlConnection db = new SqlConnection(Cnx))

    {
        db.Open();

        string actualiser = "SELECT * FROM Lot";
```

```
SqlCommand refreshCmd = new SqlCommand(actualiser, db);
```

```
SqlDataReader fiche = refreshCmd.ExecuteReader();
```

```
dataGridView1.Rows.Clear();
```

```
while (fiche.Read())
```

```
{          dataGridView1.Rows.Add(
```

```
fiche["id_lot"],
```

```
fiche["date_fabrication"],
```

```
fiche["date_peremption"],
```

```
fiche["id_produit"]);          }
```

```
fiche.Close();          } }
```

3. Code de bouton Enregistrer

```
private void button7_Click(object sender, EventArgs e)
```

```
{          using (SqlConnection db = new SqlConnection(Cnx))
```

```
{          db.Open();
```

```
string Cmd = "insert into Lot (id_lot, date_fabrication , date_peremption, id_produit) values  
( @idl, @df ,@dp,@idp) ";
```

```
SqlCommand insrt = new SqlCommand(Cmd, db);
```

```
insrt.Parameters.Add("@idl", SqlDbType.Int).Value = int.Parse(IDL.Text.Trim());
```

```

insrt.Parameters.Add("@df", SqlDbType.DateTime).Value = dateTimePicker1.Value;

insrt.Parameters.Add("@dp", SqlDbType.DateTime).Value = dateTimePicker2.Value;

insrt.Parameters.Add("@idp", SqlDbType.Int).Value = int.Parse(IDP.Text.Trim());

insrt.ExecuteNonQuery();

SqlCommand cmdAudit = new SqlCommand("prc_Audit_Lot", db);

cmdAudit.CommandType = CommandType.StoredProcedure;

cmdAudit.Parameters.AddWithValue("@Action", "INSERT");

cmdAudit.Parameters.AddWithValue("@id_lot",int.Parse(IDL.Text));

cmdAudit.Parameters.AddWithValue("@id_produit", int.Parse(IDP.Text));

cmdAudit.Parameters.AddWithValue("@date_fabrication", dateTimePicker1.Value);

cmdAudit.Parameters.AddWithValue("@date_peremption", dateTimePicker2.Value);

cmdAudit.Parameters.AddWithValue("@id_utilisateur", SecurityContext.IdUtilisateur);

cmdAudit.ExecuteNonQuery();

string actualiser = "SELECT * FROM Lot";

SqlCommand refreshCmd = new SqlCommand(actualiser, db);

SqlDataReader fiche = refreshCmd.ExecuteReader();

dataGridView1.Rows.Clear();

```

```

while (fiche.Read())

{
    dataGridView1.Rows.Add(

fiche["id_lot"],

fiche["date_fabrication"],

fiche["date_peremption"],

fiche["id_produit"]);
}

fiche.Close();

MessageBox.Show("Lot Bien Ajouter"); }

```

4. Code bouton Rechercher

```

private void button3_Click(object sender, EventArgs e)
{
    using (SqlConnection db = new SqlConnection(Cnx))
    {
        db.Open();

string srch = "SELECT * FROM Lot where id_lot =@idl and id_produit = @idp";
SqlCommand cmd = new SqlCommand(srch, db);
cmd.Parameters.Add("@idl", SqlDbType.Int).Value = int.Parse(IDL.Text);
cmd.Parameters.Add("@idp", SqlDbType.Int).Value = int.Parse(IDP.Text);
SqlDataReader fiche = cmd.ExecuteReader();

dataGridView1.Rows.Clear();
while (fiche.Read())
{
    dataGridView1.Rows.Add(
fiche["id_lot"],
fiche["date_fabrication"],
fiche["date_peremption"],
fiche["id_produit"]);
}
fiche.Close();
}
}

```

5. Code bouton Modifier

```

private void button4_Click(object sender, EventArgs e)

{
    using (SqlConnection db = new SqlConnection(Cnx))

    {
        db.Open();

        if (string.IsNullOrEmpty(IDP.Text) || string.IsNullOrEmpty(IDL.Text))

        {
            MessageBox.Show("Veuillez remplir tous les champs obligatoires!");

            return;
        }

        if (!int.TryParse(IDL.Text.Trim(), out idlot))

        {
            MessageBox.Show("ID Lot invalide !");

            return;
        }

        string Cmd = " update Lot set date_fabrication = @DF , date_peremption =
        @DP where id_lot = @idl and id_produit = @idp ";

        SqlCommand updt = new SqlCommand(Cmd, db);

        updt.Parameters.Add("@idl", SqlDbType.Int).Value =
        int.Parse(IDL.Text.Trim());

        updt.Parameters.Add("@DF", SqlDbType.DateTime).Value =
        dateTimePicker1.Value;

        updt.Parameters.Add("@DP", SqlDbType.DateTime).Value =
        dateTimePicker2.Value;

        updt.Parameters.Add("@idp", SqlDbType.Int).Value =
        int.Parse(IDP.Text.Trim());
    }
}

```

```
updt.ExecuteNonQuery();
```

```
SqlCommand cmdAudit = new SqlCommand("prc_Audit_Lot", db);
```

```
cmdAudit.CommandType = CommandType.StoredProcedure;
```

```
cmdAudit.Parameters.AddWithValue("@Action", "UPDATE");
```

```
cmdAudit.Parameters.AddWithValue("@id_lot", int.Parse(IDL.Text));
```

```
cmdAudit.Parameters.AddWithValue("@id_produit", int.Parse(IDP.Text));
```

```
cmdAudit.Parameters.AddWithValue("@date_fabrication",  
dateTimePicker1.Value);
```

```
cmdAudit.Parameters.AddWithValue("@date_peremption",  
dateTimePicker2.Value);
```

```
cmdAudit.Parameters.AddWithValue("@id_utilisateur",  
SecurityContext.IdUtilisateur);
```

```
cmdAudit.ExecuteNonQuery();
```

```
string actualiser = "SELECT * FROM Lot";
```

```
SqlCommand refreshCmd = new SqlCommand(actualiser, db);
```

```
SqlDataReader fiche = refreshCmd.ExecuteReader();
```

```
dataGridView1.Rows.Clear();
```

```
while (fiche.Read())           {
```

```
dataGridView1.Rows.Add(
```



```

        fiche["id_lot"],

        fiche["date_fabrication"],

        fiche["date_peremption"],

        fiche["id_produit"]);          }

    fiche.Close();

    MessageBox.Show("Lot Bien Modifier"); }

```

6. Code bouton supprimer

```

private void button5_Click(object sender, EventArgs e)

{
    using (SqlConnection db = new SqlConnection(Cnx))

    {
        db.Open();

        if (string.IsNullOrEmpty(IDP.Text) || string.IsNullOrEmpty(IDL.Text))

        {
            MessageBox.Show("Veuillez remplir tous les champs obligatoires !");

            return;
        }

        if (!int.TryParse(IDL.Text.Trim(), out idlot))

        {
            MessageBox.Show("ID Lot invalide !");

            return;
        }

        string deleteCmd = "DELETE FROM Lot

```

```
WHERE id_lot = @idl and id_produit = @idp ";
```

```
SqlCommand Cmd = new SqlCommand(deleteCmd, db);
```

```
Cmd.Parameters.Add("@idl", SqlDbType.Int).Value = idlot;
```

```
Cmd.Parameters.Add("@idp", SqlDbType.Int).Value = int.Parse(IDP.Text);
```

```
int rows = Cmd.ExecuteNonQuery();
```

```
if (rows > 0)
```

```
{      SqlCommand cmdAudit = new SqlCommand("prc_Audit_Lot", db);
```

```
cmdAudit.CommandType = CommandType.StoredProcedure;
```

```
cmdAudit.Parameters.AddWithValue("@Action", "DELETE");
```

```
cmdAudit.Parameters.AddWithValue("@id_lot", int.Parse(IDL.Text));
```

```
cmdAudit.Parameters.AddWithValue("@id_produit", int.Parse(IDP.Text));
```

```
cmdAudit.Parameters.AddWithValue("@date_fabrication", dateTimePicker1.Value);
```

```
cmdAudit.Parameters.AddWithValue("@date_peremption", dateTimePicker2.Value);
```

```
cmdAudit.Parameters.AddWithValue("@id_utilisateur", SecurityContext.IdUtilisateur);
```

```
cmdAudit.ExecuteNonQuery();
```

```
MessageBox.Show("Lot Bien supprime");
```

```
string actualiser = "SELECT * FROM Lot";
```

```

SqlCommand refreshCmd = new SqlCommand(actualiser, db);

SqlDataReader fiche = refreshCmd.ExecuteReader();

dataGridView1.Rows.Clear();

while (fiche.Read())

{
    dataGridView1.Rows.Add(

fiche["id_lot"],

fiche["date_fabrication"],

fiche["date_peremption"],

fiche["id_produit"]);    }

fiche.Close();    }

else    {

MessageBox.Show("Aucun Lot trouvé avec cet ID");    }    }    }

```

7. Code de bouton fermer

```

private void button6_Click(object sender, EventArgs e)

{

this.Close();    }

```

➤ Code de Procedure Stockées de tables Lot

```

CREATE PROCEDURE prc_Audit_Lot

@Action NVARCHAR(20),

@id_lot INT,

@id_produit INT,

@date_fabrication DATETIME = NULL,

@date_peremption DATETIME = NULL,

@Ancienne_date_fabrication DATETIME = NULL,

@Ancienne_date_peremption DATETIME = NULL,

@id_utilisateur INT

AS BEGIN

SET NOCOUNT ON;

DECLARE @nom_utilisateur NVARCHAR(80);

SELECT @nom_utilisateur = nom_utilisateur

FROM Utilisateur

WHERE id_utilisateur = @id_utilisateur;

INSERT INTO Audit (

id_cible, nom_table, type_op,

```

Date_op, id_utilisateur, id_utilisateur_connecte, nom_utilisateur,

ancienne_valeur, nouvelle_valeur

) VALUES (

'id Lot :' + CAST(@id_lot AS NVARCHAR(80)) + ' id produit :' + CAST(@id_produit AS NVARCHAR(80)),

'Lot',

CASE

WHEN @Action = 'INSERT' THEN 'Insertion'

WHEN @Action = 'UPDATE' THEN 'Modification'

WHEN @Action = 'DELETE' THEN 'Suppression'

END,

GETDATE(),

@id_utilisateur,

@id_utilisateur,

@nom_utilisateur,

CASE

WHEN @Action = 'UPDATE' THEN

'date fabrication :' + ISNULL(CONVERT(NVARCHAR(80), @Ancienne_date_fabrication),

120), ") +

' date peremption :' + ISNULL(CONVERT(NVARCHAR(80),
@Ancienne_date_peremption, 120), ")

WHEN @Action = 'DELETE' THEN

'date fabrication :'

+ ISNULL(CONVERT(NVARCHAR(80), @date_fabrication, 120), ") +

' date peremption :' + ISNULL(CONVERT(NVARCHAR(80), @date_peremption, 120), ")

ELSE NULL END,

CASE WHEN @Action = 'INSERT' THEN

'date fabrication :' + ISNULL(CONVERT(NVARCHAR(80), @date_fabrication, 120), ") +

' date peremption :' + ISNULL(CONVERT(NVARCHAR(80), @date_peremption, 120), ")

WHEN @Action = 'UPDATE' THEN

'date fabrication :' + ISNULL(CONVERT(NVARCHAR(80), @date_fabrication, 120), ") +

' date peremption :' + ISNULL(CONVERT(NVARCHAR(80), @date_peremption, 120), ")

ELSE NULL

END); END;

F. Interfaces D'inventaire

Description :

Cette interface permet de gérer les inventaires et comparer stock théorique et stock réel.

	id_inventaire	date_inventaire	remarque	id_entrepot	id_produit	id_utilisateur
*						

Cette table est liée à une procédure stockée d'audit qui trace les insertions et suppressions , modifications.

G. Interface Ligne de Vente.

Description :

Cette interface permet de gérer les lignes de vente (produit, lot, quantité, prix).

	id_vente	id_produit	id_lot	prix_Vente	Quantite
*					

**Cette table est liée à une procédure stockée d'audit
qui trace les insertions et suppressions,
modifications.**

H. Interface Ligne D'achat

Description :

Le module Ligne_Achat automatiquement toutes les opérations critiques (INSERT, UPDATE, DELETE) effectuées sur les tables principales. Il garantit la traçabilité des actions .

Ligne_Achat

ID Commande :

ID Produit :

ID Lot :

Prix Achat :

Quantité :

Nouveau

Actualiser

Enregistrer

Rechercher

Modifier

Supprimer

Fermer

	id_produit	id_commande	id_lot	Quantite	Prix_Achat
*					

Cette table est liée à une procédure stockée d'audit qui trace les insertions et suppressions, modifications.

I. Interface de gestion du paiement

Description :

Cette interface permet de gérer les paiements liés aux ventes (date, mode de paiement)..

Paiement

ID_Paiement :

ID_Vente :

Date_Paiement : dimanche 5 juillet 2026

Mode_paiement :

	id_paiement	date_paiement	mode_paiement	id_vente
*				

Cette table est liée à une procédure stockée d'audit qui trace les insertions et suppressions , modifications.

J. Interface Ligne de Stock

Description :

Cette interface permet de gérer les stocks par emplacement.

The interface 'Ligne_Stock' contains the following elements:

- Input fields for 'ID Produit', 'ID Emplacement', and 'Quantite'.
- A table with the following structure:

	id_produit	id_emplacement	Quantite
*			

- A vertical stack of buttons on the right side: 'Nouveau' (green), 'Actualiser' (orange), 'Enregistrer' (blue), 'Rechercher' (purple), 'Modifier' (blue), 'Supprimer' (red), and 'Fermer' (grey).

Cette table est liée à une procédure stockée d'audit qui trace les insertions et suppressions , modifications.

K. Interface Ligne d'inventaire

Description :

Cette interface permet de comparer le stock théorique et le stock réel lors des inventaires.

	id_produit	id_inventaire	stock_theorique	stock_reel
*				

Cette table est liée à une procédure stockée d'audit qui trace les insertions et suppressions, modifications.

L. Interface Produit

Description :

Cette interface permet de gérer les produits (désignation, description, prix, seuil d’alerte, quantité, catégorie).

The screenshot shows a window titled 'Produit' with a light gray background. On the left, there are six input fields with labels: 'ID Produit :', 'ID Catégorie :', 'Designation :', 'Prix Unitaire :', 'Description :', and 'Quantité :'. To the right of these fields is a vertical stack of seven buttons: 'Nouveau' (green), 'Actualiser' (orange), 'Enregistrer' (blue), 'Rechercher' (purple), 'Modifier' (blue), 'Supprimer' (red), and 'Fermer' (gray). Below the input fields is a table with 8 columns: 'id_produit', 'designation', 'prix_unitaire', 'seuil_alerte', 'description_p', 'id_categorie', and 'quantité'. The first row of the table has a '*' in the first column and empty cells for the others. The rest of the table is obscured by a large gray rectangle.

	id_produit	designation	prix_unitaire	seuil_alerte	description_p	id_categorie	quantité
*							

Cette table a un trigger qui manipule le statut de Produit.

Exemple de crud Normal :

Code de bouton Nouveau

```
private void button1_Click(object sender, EventArgs e)
{
    idp.Clear();
    idc.Clear();
    d.Clear();
    p.Clear();
    desc.Clear();
    q.Clear();
    idp.Select();
}
```

1. Code de bouton Actualiser

```
private void button2_Click(object sender, EventArgs e)
{
    using (SqlConnection db = new SqlConnection(Cnx))
    {
        db.Open();

        string actualiser = "SELECT * FROM Produit";
        SqlCommand refreshCmd = new SqlCommand(actualiser, db);
        SqlDataReader fiche = refreshCmd.ExecuteReader();

        dataGridView1.Rows.Clear();
        while (fiche.Read())
        {
            dataGridView1.Rows.Add(
                fiche["id_produit"],
                fiche["designation"],
                fiche["prix_unitaire"],
                fiche["seuil_alerte"],
                fiche["description_p"],
                fiche["id_categorie"],
                fiche["quantité"]);
        }
        fiche.Close();
    }
}
```

2. Code de bouton Enregistrer

```
private void button7_Click(object sender, EventArgs e)
{
    using (SqlConnection db = new SqlConnection(Cnx))
    {
        db.Open();

        string Cmd = "insert into Produit(id_produit, designation, prix_unitaire, description_p, id_categorie, quantité) values ( @IDP, @DESG, @PU, @DESC, @IDC, @Q) ";
        SqlCommand insrt = new SqlCommand(Cmd, db);
        insrt.Parameters.Add("@IDP", SqlDbType.Int).Value = int.Parse(idp.Text.Trim());
        insrt.Parameters.Add("@DESG", SqlDbType.VarChar, 80).Value = d.Text.Trim();
        insrt.Parameters.Add("@PU", SqlDbType.Decimal).Value = decimal.Parse(p.Text.Trim());
        insrt.Parameters.Add("@DESC", SqlDbType.VarChar, 200).Value = desc.Text.Trim();
        insrt.Parameters.Add("@IDC", SqlDbType.VarChar, 200).Value =
            int.Parse(idc.Text.Trim());
        insrt.Parameters.Add("@Q", SqlDbType.Int).Value = int.Parse(q.Text.Trim());
    }
}
```

```
insrt.ExecuteNonQuery();
```

```
string actualiser = "SELECT * FROM Produit";  
SqlCommand refreshCmd = new SqlCommand(actualiser, db);  
SqlDataReader fiche = refreshCmd.ExecuteReader();
```

```
dataGridView1.Rows.Clear();  
while (fiche.Read())  
{ dataGridView1.Rows.Add(  
fiche["id_produit"],  
fiche["designation"],  
fiche["prix_unitaire"],  
fiche["seuil_alerte"],  
fiche["description_p"],  
fiche["id_categorie"],  
fiche["quantité"]); }  
fiche.Close();
```

```
MessageBox.Show("produit Bien Ajouter"); } }
```

Code de bouton Modifier

```
private void button4_Click(object sender, EventArgs e)  
{ using (SqlConnection db = new SqlConnection(Cnx))  
{ db.Open();  
f (string.IsNullOrEmpty(idp.Text) ||  
string.IsNullOrEmpty(idc.Text) ||  
string.IsNullOrEmpty(d.Text) ||  
string.IsNullOrEmpty(p.Text) ||  
string.IsNullOrEmpty(desc.Text) ||  
string.IsNullOrEmpty(q.Text))  
{ MessageBox.Show("Veuillez remplir tous les champs obligatoires !");  
return;  
}  
if (!int.TryParse(idc.Text.Trim(), out idClient) {  
MessageBox.Show("ID client invalide !");  
return; }  
string Cmd = " update Produit set designation = @D ,prix_unitaire = @PU ,  
description_p = @DESC ,quantité = @Q  
where id_produit = @IDP AND id_categorie = @IDC ";  
  
SqlCommand updt = new SqlCommand(Cmd, db);  
updt.Parameters.Add("@IDP", SqlDbType.Int).Value = int.Parse(idp.Text.Trim());  
updt.Parameters.Add("@IDC", SqlDbType.VarChar, 200).Value =  
int.Parse(idc.Text.Trim());  
updt.Parameters.Add("@D", SqlDbType.VarChar, 80).Value = d.Text.Trim();
```

```

updt.Parameters.Add("@PU", SqlDbType.Decimal).Value =
decimal.Parse(p.Text.Trim());
updt.Parameters.Add("@DESC", SqlDbType.VarChar, 200).Value = desc.Text.Trim();
updt.Parameters.Add("@Q", SqlDbType.Int).Value = int.Parse(q.Text.Trim());
updt.ExecuteNonQuery();

```

```

string actualiser = "SELECT * FROM Produit";
SqlCommand refreshCmd = new SqlCommand(actualiser, db);
SqlDataReader fiche = refreshCmd.ExecuteReader();

```

```

dataGridView1.Rows.Clear();
while (fiche.Read())
{
    dataGridView1.Rows.Add(
        fiche["id_produit"],
        fiche["designation"],
        fiche["prix_unitaire"],
        fiche["seuil_alerte"],
        fiche["description_p"],
        fiche["id_categorie"],
        fiche["quantité"]);
}
fiche.Close();
MessageBox.Show("produit Bien Modifier");
}
}

```

Code de bouton Rechercher

```

private void button3_Click(object sender, EventArgs e)
{
    using (SqlConnection db = new SqlConnection(Cnx))
    {
        db.Open();
    }

```

```

string actualiser = "select * from PRODUIT where id_produit = @IDP and id_categorie = @IDC ";
SqlCommand Cmd = new SqlCommand(actualiser, db);
Cmd.Parameters.AddWithValue("@IDP", idp.Text.Trim());
Cmd.Parameters.AddWithValue("@IDC", idc.Text.Trim());
SqlDataReader Fiche = Cmd.ExecuteReader();

```

```

dataGridView1.Rows.Clear();
while (Fiche.Read())
{
    dataGridView1.Rows.Add(
        Fiche["id_produit"],
        Fiche["designation"],
        Fiche["prix_unitaire"],
        Fiche["seuil_alerte"],
        Fiche["description_p"],
        Fiche["id_categorie"],
        Fiche["quantité"]);
}
Fiche.Close();
}
}

```


Code de bouton Supprimer

```
private void button5_Click(object sender, EventArgs e)
{
    using (SqlConnection db = new SqlConnection(Cnx))
    {
        db.Open();

        int idproduit;
        int idcategorie;
        if (!int.TryParse(idc.Text.Trim(), out idproduit) || !int.TryParse(idc.Text.Trim(), out idcategorie))
        {
            MessageBox.Show("ID produit ou de catégorie invalide !");
            return;
        }

        string deleteCmd = "DELETE FROM Produit WHERE id_produit = @IDP AND id_categorie = @IDC";
        SqlCommand Cmd = new SqlCommand(deleteCmd, db);
        Cmd.Parameters.Add("@IDP", SqlDbType.Int).Value = idproduit;
        Cmd.Parameters.Add("@IDC", SqlDbType.Int).Value = idcategorie;
        int rows = Cmd.ExecuteNonQuery();
        if (rows > 0)
        {
            MessageBox.Show("produit Bien Supprimé");
            string actualiser = "SELECT * FROM produit";
            SqlCommand refreshCmd = new SqlCommand(actualiser, db);
            SqlDataReader fiche = refreshCmd.ExecuteReader();

            dataGridView1.Rows.Clear();
            while (fiche.Read())
            {
                dataGridView1.Rows.Add(
                    fiche["id_produit"],
                    fiche["designation"],
                    fiche["prix_unitaire"],
                    fiche["seuil_alerte"],
                    fiche["description_p"],
                    fiche["id_categorie"],
                    fiche["quantité"]);
            }

            fiche.Close();
        }
        else
        {
            MessageBox.Show("Aucun produit trouvé avec cet ID");
        }
    }
}
```

Code de bouton Fermer

```
private void button6_Click(object sender, EventArgs e)
{
    this.Close();
}
```

M. Interface de Vente

Description :

Cette interface permet de gérer les ventes effectuées (date, client, utilisateur, mode de paiement).

The interface is titled 'Vente' and contains the following elements:

- Four input fields on the left:
 - ID_Vente :
 - ID_utilisateur :
 - ID_bon_livraison ::
 - id_client :
- A table with the following columns: id_vente, date_vente, id_utilisateur, id_bon_livraison, id_client. The first row contains an asterisk (*) in the first column.
- A vertical stack of seven buttons on the right:
 - Nouveau (green)
 - Actualiser (orange)
 - Enregistrer (blue)
 - Rechercher (purple)
 - Modifier (blue)
 - Supprimer (red)
 - Fermer (gray)

Cette table possède des triggers qui tracent les ventes (ajout, modification, suppression).

N. Interface Bon de Livraison

Description :

Cette interface permet de gérer les bons de livraison associés aux ventes. *Note: Cette table ne possède pas de trigger ou procédure spécifique.*

Bon_Livraison

ID Bon_Livraison :

Nouveau

Date_Livraison :

lundi 6 juillet 2026

Actualiser

Adresse :

Enregistrer

	id_bon_Livraison	date_livraison	adresse
*			

Rechercher

Modifier

Supprimer

Femer

O. Interface Comande d'achat

Description :

Cette interface permet de gérer les commandes passées auprès des fournisseurs.

Commande_Achat

ID_Commande :

Statut :

ID_Utilisateur :

ID_Fournisseur :

	id_commande	date_commande	statut	id_utilisateur	id_fournisseur
*					

Nouveau

Actualiser

Enregistrer

Rechercher

Modifier

Supprimer

Fermer

Cette table est associée à une procédure stockée qui gère l'audit des commandes.

P. Interface Catégorie

Description :

Le module Catégorie enregistre automatiquement toutes les opérations critiques (INSERT, UPDATE, DELETE) effectuées sur les tables principales. Il garantit la traçabilité des actions .

The screenshot shows a window titled 'catégorie' with a light gray background. On the left, there are two input fields: 'ID_catégorie :' and 'Nom_Catégorie :', each followed by a white text box. Below these is a table with two columns: 'id_categorie' and 'nom_categorie'. The first row of the table has a small asterisk in the first column and empty cells in the others. To the right of the table and input fields is a vertical stack of six buttons: 'Nouveau' (green), 'Actualiser' (orange), 'Enregistrer' (blue), 'Rechercher' (purple), 'Modifier' (blue), and 'Supprimer' (red). At the bottom right is a 'Femer' button (gray).

	id_categorie	nom_categorie
*		

Elle ne possède pas de procédure stockée ni de trigger.

Q. Interface Fournisseur

Description :

Cette interface contient les informations relatives aux fournisseurs (nom, adresse, téléphone, email).

The screenshot shows a web application window titled "Fournisseur". It contains several input fields for data entry:

- ID Fournisseur :
- Nom Complet :
- Mail :
- Adresse :
- Telephone :

Below these fields is a table with the following structure:

	id_Fournisseur	nom_complet	mail	adresse	telephone
*					

On the right side of the interface, there is a vertical column of buttons:

- Nouveau (Green)
- Actualiser (Orange)
- Enregistrer (Blue)
- Rechercher (Purple)
- Modifier (Blue)
- Supprimer (Red)
- Fermer (Grey)

Aucun trigger ou procédure n'est associé directement à cette table.

R. Interface Fournir

Description :

Cette interface permet de gérer la relation entre les fournisseurs et les produits fournis.

The interface 'Fournir' contains the following elements:

- Input field: ID Produit
- Input field: ID Fournisseur:
- Input field: Quantité :
- Table with columns: id_produit, id_fournisseur, quantite. The first row contains an asterisk (*).
- Buttons: Nouveau (green), Actualiser (orange), Enregistrer (blue), Rechercher (purple), Modifier (blue), Supprimer (red), Fermer (gray).

Cette table ne possède pas de trigger ou procédure spécifique.

S. Interface Entrepot

Description :

Cette interface permet de gérer les entrepôts physiques où les produits sont stockés..

The screenshot shows a window titled "Entrepot" with a light gray background. On the left, there are three input fields with labels: "ID entrepot :", "Nom magasin :", and "Adresse :". Below these is a table with three columns: "id_entrepot", "nom_magasin", and "adresse". The first row of the table contains an asterisk (*) in the first column. To the right of the table and input fields is a vertical column of seven buttons: "Nouveau" (green), "Actualiser" (orange), "Enregistrer" (blue), "Rechercher" (blue), "Modifier" (purple), "Supprimer" (red), and "Fermer" (gray).

	id_entrepot	nom_magasin	adresse
*			

Elle sert uniquement à la localisation et ne contient pas de trigger ou procédure.

T. Interface Emplacement

Description :

Le module Emplacement eCette interface définit les emplacements internes des entrepôts (rayons, zones).

	id_emplacement	code_emplacement	id_entrepot
*			

Aucun trigger ou procédure n'est lié à cette table.

M. Interface Audit

Description :

Cette interface centralise toutes les opérations tracées par les triggers et procédures des autres tables.

Audit									
id_audit	id_cible	nom_table	type_op	Date_op	nom_utilisateur	ancienne_valeur	nouvelle_valeur	id_utilisateur	id_utilisateur_conr
245	id produit :1 id co...	Ligne Achat	Suppression	01/07/2026 05:3...	amine	Quantite :0 Prix_...		1	1
246	id inventaire :1 id ...	Ligne_inventaire	Suppression	01/07/2026 05:3...	amine	stock théorique : ...		1	1
247	id inventaire :1	Inventaire	Suppression	01/07/2026 05:3...	amine	date juil 1 2026 ...		1	1
248	id produit :2 id co...	Ligne Achat	Suppression	01/07/2026 05:5...	amine	Quantite :0 Prix_...		1	1
249	id produit :2 id co...	Ligne Achat	Suppression	01/07/2026 05:5...	amine	Quantite :0 Prix_...		1	1
250	id produit :1 id co...	Ligne Achat	Suppression	01/07/2026 05:5...	amine	Quantite :0 Prix_...		1	1
251	id Lot :1 id produi...	Lot	Suppression	01/07/2026 06:1...	amine	date fabrication : ...		1	1
252	id Lot :2 id produi...	Lot	Suppression	01/07/2026 06:1...	amine	date fabrication : ...		1	1
*									

Actualiser

Fermer

C'est La table qui stock les Résultat des déclencheurs et des procédures stockées

N. Interface Role

Description :

Cette interface permet de définir les rôles des utilisateurs (Administrateur, Magasinier, Responsable).

The screenshot shows a window titled "Role" with a standard Windows-style title bar (minimize, maximize, close buttons). Inside the window, there are two input fields on the left: "ID_Role :" and "Libellée :". To the right of these fields is a vertical stack of seven buttons: "Nouveau" (green), "Actualiser" (orange), "Enregistrer" (blue), "Rechercher" (purple), "Modifier" (blue), "Supprimer" (red), and "Fermer" (gray). Below the input fields is a table with two columns: "id_role" and "libelle". The table contains three rows of data and a header row. The first row is highlighted in blue.

	id_role	libelle
▶	1	Administrateur
	2	Magasinier
	3	Responsable
*		

Cette table ne possède pas de trigger ou procédure spécifique.

O. Interface Client

Description ;

Cette interface permet de gérer les informations des clients (nom, adresse, téléphone, email).

Client

ID Client :

Nom Client :

Telephone :

Mail :

Adresse :

Nouveau

Actualiser

Enregistrer

Rechercher

Modifier

Supprimer

Fermer

	id_client	nom_complet	telephone	email	adresse
*					

Cette table est liée à une procédure stockée qui enregistre les opérations d'ajout et de modification dans le module d'audit.

Résultats et avantages

La mise en place de ce système de gestion de stock a apporté plusieurs résultats concrets et avantages :

- **Fiabilité accrue des données** : grâce aux procédures stockées et aux triggers d'audit, chaque opération est tracée, ce qui réduit considérablement les erreurs et garantit la cohérence des informations.
- **Optimisation des processus** : la centralisation des données dans SQL Server et l'utilisation d'interfaces C# WinForms ont permis de fluidifier la gestion des ventes, achats et inventaires.
- **Gain de temps** : les opérations CRUD sont simplifiées et automatisées, ce qui accélère le travail des utilisateurs et diminue les tâches répétitives.
- **Traçabilité et sécurité** : chaque action (INSERT, UPDATE, DELETE) est enregistrée avec l'utilisateur, la date et les valeurs modifiées, assurant une transparence totale.
- **Évolutivité** : la conception basée sur Merise et UML, combinée aux technologies Desktop, offre une solution robuste et adaptable aux futurs besoins de l'entreprise.

Conclusion et Perspective :

Dans une optique d'amélioration continue, plusieurs évolutions peuvent être envisagées :

- **Développement d'une version web ou mobile pour un accès distant.**
- **Intégration d'un module de statistiques et de reporting avancé.**
- **Mise en place d'un système de notifications automatiques pour les seuils d'alerte.**
- **Extension vers la gestion multi-entrepôts et multi-utilisateurs avec des rôles plus détaillés.**

Webographie :

- **Documentation officielle Microsoft Visual Studio :**
<https://visualstudio.microsoft.com> (visualstudio.microsoft.com in Bing)
- **Documentation .NET Framework :**
<https://learn.microsoft.com/dotnet/framework>
- **Documentation ADO.NET :**
<https://learn.microsoft.com/dotnet/framework/data/adonet> (learn.microsoft.com in Bing)
- **Documentation SQL Server :**
<https://learn.microsoft.com/sql>
- **Outil UML (diagrammes) :**
<https://app.diagrams.net>